שאלות ותשובות למבחן במבני נתונים ואלגוריתמים

שאלות ותשובות למבחן במבני נתונים ואלגוריתמים

סיכום זה מציג מאגר שאלות תרגול למבחן במבני נתונים ואלגוריתמים, המסודרות לפי שלוש רמות קושי: קלה, בינונית וקשה. לכל שאלה מצורף פתרון מפורט, הערות מרצה, ודגשים למניעת טעויות נפוצות במבחן.

תוכן עניינים

1. שאלות ברמה קלה (Basic/Easy)
   - שאלה 1: ניתוח סיבוכיות אסימפטוטית
   - שאלה 2: פתרון משוואת רקורסיה
   - שאלה 3: פעולות בערימה בינארית
   - שאלה 4: שחזור עץ חיפוש בינארי (BST)
2. שאלות ברמה בינונית (Intermediate/Medium)
   - שאלה 5: ניתוח לשיעורין של מחסנית מרוקנת
   - שאלה 6: איזון עצי AVL
   - שאלה 7: גיבוב מושלם FKS
   - שאלה 8: תכנון דינמי - תת-סדרה עולה ארוכה ביותר (LIS)
3. שאלות ברמה קשה (Advanced/Hard)
   - שאלה 9: תכנון דינמי - וריאציית משחק השטרות
   - שאלה 10: מבנה קבוצות זרות (Union-Find) עם משקלים
   - שאלה 11: אלגוריתם אימות עץ AVL בשידור ליניארי
   - שאלה 12: ניתוח פיצולים במקרה הגרוע בעץ B-Tree

שאלות ברמה קלה (Basic/Easy)

שאלה 1: ניתוח סיבוכיות אסימפטוטית

הוכח או הפרך את הטענה הבאה על פי ההגדרות הפורמליות של החסמים האסימפטוטיים:
בהינתן שתי פונקציות חיוביות ממש $f(n), g(n)$, אם מתקיים $f(n) \in O(g(n))$, אזי בהכרח מתקיים:

$$2^{f(n)} \in O(2^{g(n)})$$

פתרון מפורט:

הטענה **אינה נכונה** (הפרכה).
נציג דוגמה נגדית. נבחר את הפונקציות הבאות:

$$f(n) = 2n$$

$$g(n) = n$$

ברור כי $f(n) \in O(g(n))$ מכיוון שקיים קבוע $c = 2$ ו-$n_0 = 1$ כך שלכל $n \geq n_0$ מתקיים:

$$f(n) = 2n \leq 2 \cdot n = 2 \cdot g(n)$$

נבדוק כעת את החזקות של 2 עבור פונקציות אלו:

$$2^{f(n)} = 2^{2n} = (2^2)^n = 4^n$$

$$2^{g(n)} = 2^n$$

נהפוך זאת לאי-שוויון אסימפטוטי ונניח בשלילה כי $2^{f(n)} \in O(2^{g(n)})$. על פי הגדרת חסם עליון אסימפטוטי, צריכים להצליח למצוא קבועים $c > 0$ ו-$n_0 \geq 0$ כך שלכל $n \geq n_0$ יתקיים:

$$4^n \leq c \cdot 2^n$$

נחלק את שני אגפי האי-שוויון ב-2^n (מותר מכיוון ש-$2^n > 0$ לכל n):

$$\frac{4^n}{2^n} \leq c \implies \left(\frac{4}{2}\right)^n \leq c \implies 2^n \leq c$$

הגענו לאי-שוויון $2^n \leq c$. אי-שוויון זה אינו יכול להתקיים לכל $n \geq n_0$ עבור שום קבוע c סופי, מכיוון שהפונקציה 2^n שואפת לאינסוף כאשר n שואף לאינסוף, ולכן היא תעבור כל קבוע c שנבחר.
קיבלנו סתירה, ולכן הטענה מופרכת.

⚠ זהירות - טעות נפוצה: חוקי חזקות אסימפטוטיים

סטודנטים רבים נוטים לחשוב בטעות שניתן להפעיל פונקציות מונוטוניות על שני אגפים של יחס אסימפטוטי (כמו חזקה או לוגריתם). בעוד שבלוגריתמים זה נכון בתנאים מסוימים (אם הפונקציות שואפות לאינסוף), בחזקות זה כמעט תמיד שגוי, משום שהפרש קבוע במעריך מתורגם למכפיל קבוע בבסיס, אך הפרש של פונקציה (כמו $2n - n = n$) מתורגם למכפיל שאינו קבוע אלא תלוי ב-n (2^n).

שאלה 2: פתרון משוואת רקורסיה

פתור את משוואת הרקורסיה הבאה ומצא חסם אסימפטוטי הדוק (Θ) בעזרת משפט המאסטר:

$$T(n) = 4T\left(\frac{n}{2}\right) + n^2\sqrt{n}$$

פתרון מפורט:

משוואת הרקורסיה היא מהצורה הסטנדרטית של משפט המאסטר:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

כאשר:

- $a = 4$
- $b = 2$
- $f(n) = n^2\sqrt{n} = n^{2.5}$

נחשב את איבר השוואה הדומיננטי של העץ $n^{\log_b a}$:

$$n^{\log_2 4} = n^2$$

כעת נשווה אסימפטוטית בין פונקציית העבודה הישירה $f(n) = n^{2.5}$ לבין $n^{\log_b a} = n^2$:
נבחין כי $f(n)$ גדלה מהר יותר. נבדוק אם מתקיים מקרה 3 של משפט המאסטר.
עלינו להראות כי $f(n) \in \Omega(n^{\log_b a + \epsilon})$ עבור קבוע $\epsilon > 0$.
נבחר $\epsilon = 0.5$:

$$n^{2.5} \in \Omega(n^{2+0.5}) = \Omega(n^{2.5})$$

תנאי זה מתקיים. כעת עלינו לבדוק את **תנאי הרגולריות**:
עלינו להראות שקיים קבוע $c < 1$ כך שלכל n גדול מספיק מתקיים:

$$af\left(\frac{n}{b}\right) \leq cf(n)$$

נציב את הנתונים באגף שמאל:

$$af\left(\frac{n}{b}\right) = 4f\left(\frac{n}{2}\right) = 4\left(\frac{n}{2}\right)^{2.5} = 4\frac{n^{2.5}}{2^{2.5}} = 4\frac{n^{2.5}}{2^2 \cdot \sqrt{2}} = 4\frac{n^{2.5}}{4\sqrt{2}} = \frac{1}{\sqrt{2}}n^{2.5}$$

נדרוש:

$$\frac{1}{\sqrt{2}}n^{2.5} \leq c \cdot n^{2.5} \iff c \geq \frac{1}{\sqrt{2}} \approx 0.707$$

מכיוון ש-$\frac{1}{\sqrt{2}} < 1$, נוכל לבחור קבוע c המקיים $\frac{1}{\sqrt{2}} \leq c < 1$ (למשל $c = 0.8$), ותנאי הרגולריות מתקיים לכל $n \geq 1$.
על פי מקרה 3 של משפט המאסטר, פתרון הרקורסיה נקבע על ידי עבודת השורש $f(n)$:

$$T(n) \in \Theta(f(n)) = \Theta(n^2\sqrt{n})$$

↘ טענה: מקרה 3 של משפט המאסטר

כאשר העבודה בשורש העץ גדולה משמעותית מהעבודה בעלים (בפקטור פולינומי של לפחות n^ϵ), ותנאי הרגולריות מתקיים, אזי רוב העבודה של האלגוריתם מתבצעת בקריאה הראשונה (בשורש), וכל שאר רמות הרקורסיה יחד מהוות טור הנדסי יורד שנחסם על ידי קבוע כפול השורש.

שאלה 3: פעולות בערימה בינארית

נתונה ערימת מינימום בינארית המיוצגת במערך הבא:

$$Q = [3, 8, 10, 15, 12, 20, 25]$$

1. הצג את העץ התואם למערך.
2. בצע פעולת הכנסה של המפתח 2 לערימה, והצג את שלבי התיקון מעלה (Heapify-up) ואת המערך הסופי.
3. על הערימה שנוצרה בסעיף 2, בצע פעולת הוצאת מינימום (Delete-Min), והצג את שלבי התיקון מטה (Heapify-down) ואת המערך הסופי.

פתרון מפורט:

1. ייצוג העץ המקורי:

נצייר את העץ לפי אינדקסי המערך (הבן השמאלי של i הוא $2i+1$, והבן הימני הוא $2i+2$):

```
      [3] (index 0)
     /   \
  [8]     [10] (indices 1, 2)
  /  \    /   \
[15] [12][20] [25] (indices 3, 4, 5, 6)
```

2. הכנסת המפתח 2 לערימה:

- **שלב א':** נציב את 2 בסוף המערך (במקום הפנוי הראשון בעץ כבן שמאלי של 15, באינדקס 7):
  המערך הזמני: $[3, 8, 10, 15, 12, 20, 25, 2]$
- **שלב ב' (Heapify-up מהאינדקס $i = 7$):**
  - ההורה של $i = 7$ נמצא באינדקס $\lfloor (7-1)/2 \rfloor = 3$ (ערכו 15).
  - מכיוון ש-$Q[7] = 2 < Q[3] = 15$, נבצע החלפה ביניהם.
  - מערך לאחר החלפה: $[3, 8, 10, 2, 12, 20, 25, 15]$, האינדקס הנוכחי של המפתח הוא $i = 3$.
  - ההורה של $i = 3$ נמצא באינדקס $\lfloor (3-1)/2 \rfloor = 1$ (ערכו 8).
  - מכיוון ש-$Q[3] = 2 < Q[1] = 8$, נבצע החלפה ביניהם.
  - מערך לאחר החלפה: $[3, 2, 10, 8, 12, 20, 25, 15]$, האינדקס הנוכחי של המפתח הוא $i = 1$.
  - ההורה של $i = 1$ נמצא באינדקס $\lfloor (1-1)/2 \rfloor = 0$ (ערכו 3).
  - מכיוון ש-$Q[1] = 2 < Q[0] = 3$, נבצע החלפה ביניהם.
  - מערך לאחר החלפה: $[2, 3, 10, 8, 12, 20, 25, 15]$, האינדקס הנוכחי של המפתח הוא $i = 0$ (הגענו לשורש, התהליך נעצר).
- **המערך הסופי לאחר הכנסה:** $[2, 3, 10, 8, 12, 20, 25, 15]$

3. הוצאת מינימום (Delete-Min) מהערימה שנוצרה:

- **שלב א':** שומרים את השורש (2). לוקחים את האיבר האחרון במערך (15, באינדקס 7) ושמים אותו בשורש באינדקס 0. מקטינים את גודל הערימה ב-1.
  המערך הזמני: $[15, 3, 10, 8, 12, 20, 25]$
- **שלב ב' (Heapify-down מאינדקס $i = 0$):**
  - הבנים של $i = 0$ הם באינדקסים $l = 1$ (ערכו 3) ו-$r = 2$ (ערכו 10).
  - הערך הקטן מבין השלישייה $\{Q[0] = 15, Q[1] = 3, Q[2] = 10\}$ הוא 3 (באינדקס 1).
  - נבצע החלפה בין אינדקס 0 לאינדקס 1.
  - מערך לאחר החלפה: $[3, 15, 10, 8, 12, 20, 25]$, האינדקס הנוכחי של המפתח המוחלק הוא $i = 1$.
  - הבנים של $i = 1$ הם באינדקסים $l = 3$ (ערכו 8) ו-$r = 4$ (ערכו 12).
  - הערך הקטן מבין השלישייה $\{Q[1] = 15, Q[3] = 8, Q[4] = 12\}$ הוא 8 (באינדקס 3).
  - נבצע החלפה בין אינדקס 1 לאינדקס 3.
  - מערך לאחר החלפה: $[3, 8, 10, 15, 12, 20, 25]$, האינדקס הנוכחי של המפתח המוחלק הוא $i = 3$.
  - הבן של $i = 3$ הוא באינדקס $l = 7$ שגדול מגודל המערך (אין בנים). התהליך נעצר.
- **המערך הסופי לאחר מחיקה:** $[3, 8, 10, 15, 12, 20, 25]$ (חזרנו בדיוק למערך המקורי).

שאלה 4: שחזור עץ חיפוש בינארי (BST)

נתון מעבר סריקה מסוג Preorder של עץ חיפוש בינארי (BST):

$$\text{Preorder} = [15, 10, 5, 12, 20, 18, 25]$$

1. שחזר את העץ הבינארי הייחודי התואם לסריקה זו.
2. רשום את סריקת ה-Postorder של העץ ששוחזר.

פתרון מפורט:

1. שחזור העץ:

בעץ חיפוש בינארי (BST), סריקה תוכית (Inorder) מחזירה תמיד את המפתחות כשהם ממוינים בסדר עולה. לפיכך, אנו יכולים לקבל את סריקת ה-Inorder של העץ על ידי מיון פשוט של מערך ה-Preorder הנתון:

$$\text{Inorder} = [5, 10, 12, 15, 18, 20, 25]$$

כעת יש לנו את סריקות ה-Preorder וה-Inorder של העץ, המאפשרות שחזור רקורסיבי חד-ערכי:

- ...

העץ המשוחזר:

```
      [15]
     /    \
  [10]    [20]
  /  \    /  \
[5]  [12][18] [25]
```

2. סריקת ה-Postorder של העץ:

סריקת Postorder מוגדרת כסריקת תת-עץ שמאלי, תת-עץ ימני, ולבסוף קודקוד השורש (שמאל \to ימין \to שורש):

- תת-עץ שמאלי של 15 מחזיר: $5, 12, 10$.
- תת-עץ ימני של 15 מחזיר: $18, 25, 20$.
- נוסיף את השורש 15 בסוף.

סריקת ה-Postorder הסופית:

$$\text{Postorder} = [5, 12, 10, 18, 25, 20, 15]$$

משפט: שחזור BST מסריקה יחידה

בעץ בינארי כללי, לא ניתן לשחזר עץ באופן חד-ערכי מתוך סריקת Preorder בלבד (נדרשות לפחות שתי סריקות, למשל Preorder ו-Inorder). לעומת זאת, בעץ חיפוש בינארי (BST) ניתן לבצע שחזור חד-ערכי מסריקת Preorder בלבד (או Postorder בלבד), מכיוון שדרישת המיון של BST מגדירה מראש ובאופן מרומז את סריקת ה-Inorder של העץ.

שאלות ברמה בינונית (Intermediate/Medium)

שאלה 5: ניתוח לשיעורין של מחסנית מרוקנת

נתונה מחסנית המכילה איברים ותומכת בפעולות `Push(x)` ו-`Pop()` הרגילות שעלותן האמיתית היא $O(1)$. כעת אנו מוסיפים פעולה חדשה: `Clear()`, המרוקנת את כל האיברים מהמחסנית בבת אחת. העלות האמיתית של פעולת `Clear()` היא ליניארית במספר האיברים k שהיו במחסנית ברגע הקריאה לה, כלומר $c = k$.
הוכח באמצעות **שיטת הפוטנציאל** כי העלות לשיעורין של כל שלוש הפעולות (`Push`, `Pop`, `Clear`) לאורך סדרת פעולות המתחילה ממחסנית ריקה היא $O(1)$.

פתרון מפורט:

נגדיר את פונקציית הפוטנציאל $\Phi(D)$ על מצב המחסנית D:

$$\Phi(D) = 2 \cdot s$$

כאשר s הוא מספר האיברים הנוכחי במחסנית.
נבדוק את תנאי הפוטנציאל:

1. עבור מחסנית ריקה בהתחלה D_0, מתקיים $s_0 = 0$, ולכן $\Phi(D_0) = 0$.
2. לכל מצב D_i של המחסנית, מספר האיברים הוא אי-שלילי ($s_i \geq 0$), ולכן מתקיים תמיד:

$$\Phi(D_i) = 2s_i \geq 0 = \Phi(D_0)$$

הפוטנציאל חוקי. כעת נחשב את העלות לשיעורין $\hat{c}_i$ עבור כל אחת מהפעולות:

א. פעולת `Push(x)` :

העלות האמיתית היא $c_i = 1$. מספר האיברים גדל ב-1: $s_i = s_{i-1} + 1$.

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 + 2(s_{i-1} + 1) - 2s_{i-1} = 1 + 2s_{i-1} + 2 - 2s_{i-1} = 3 \in O(1)$$

ב. פעולת `Pop()` :

העלות האמיתית היא $c_i = 1$ (בהנחה שהמחסנית אינה ריקה). מספר האיברים קטן ב-1: $s_i = s_{i-1} - 1$.

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 + 2(s_{i-1} - 1) - 2s_{i-1} = 1 + 2s_{i-1} - 2 - 2s_{i-1} = -1 \in O(1)$$

ג. פעולת `Clear()` :

נסמן את מספר האיברים במחסנית לפני הפעולה כ-k. העלות האמיתית היא $c_i = k$. לאחר הפעולה המחסנית מתרוקנת לחלוטין ולכן $s_i = 0$.

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = k + 2 \cdot 0 - 2k = k - 2k = -k$$

נשים לב כי העלות לשיעורין שהתקבלה היא שלילית (או אפס, אם המחסנית הייתה ריקה). מאחר ש-$k \geq 0$, מתקיים:

$$\hat{c}_i = -k \leq 0 \in O(1)$$

קיבלנו שבכל המקרים העלות לשיעורין של הפעולות חסומה מלעיל על ידי קבוע (3 עבור `Push` , -1 עבור `Pop` , ו-0 עבור `Clear`).
לפיכך, העלות לשיעורין של כל פעולה היא $O(1)$.

> **פישוט והסבר המעבר:**
>
> בשיטת הפוטנציאל, העלות לשיעורין מוגדרת כעבודה האמיתית בתוספת השינוי בפוטנציאל: $\hat{c}_i = c_i + \Delta\Phi$. בפעולות `Push` אנו "טוענים" את המערכת בפוטנציאל (משלמים מראש על מחיקות עתידיות). כאשר מגיעה פעולה יקרה כמו `Clear` , העלות האמיתית הגבוהה שלה (k) מנוטרלת לחלוטין על ידי ירידה חדה בפוטנציאל (מ-$2k$ ל-0, כלומר $\Delta\Phi = -2k$), מה שמבטיח עלות משוערכת שלילית.

שאלה 6: איזון עצי AVL

נתון עץ AVL הבא (המספרים בתוך הקודקודים מייצגים מפתחות, והערכים בסוגריים מייצגים את גורם האיזון $\text{BF} = \text{height}(L) - \text{height}(R)$):

```
      [15] (BF = 1)
     /    \
  [10]    [25] (BF = 0, BF = -1)
  /      /    \
[5]    [20]   [30] (BF=0, BF=0, BF=0)
```

1. הכנס את המפתח 18 לעץ. הצג את העץ מיד לאחר ההכנסה הפיזית ולפני תיקון האיזון, וציין באיזה קודקוד נוצרת הפרת איזון ואיזה סוג רוטציה נדרש לבצע.
2. בצע את הרוטציה והצג את העץ המאוזן הסופי שמתקבל.

פתרון מפורט:

1. הכנסת המפתח 18 לעץ ולפני איזון:

על פי חוקי BST, נחפש את מיקום ההכנסה של 18:

- $18 > 15 \implies$ פונים ימינה ל-25.
- $18 < 25 \implies$ פונים שמאלה ל-20.
- $18 < 20 \implies$ פונים שמאלה. מאחר שאין ל-20 בן שמאלי, אנו תולים את 18 כבן השמאלי שלו.

נצייר את העץ הלא מאוזן ונחשב את גורמי האיזון החדשים מלמטה למעלה לאורך מסלול ההכנסה $(15 \to 25 \to 20 \to 18)$:

- עלה חדש: $\text{BF}(18) = 0$.
- קודקוד 20: הבן השמאלי בגובה 0, הימני בגובה -1 (ריק) $\implies \text{BF}(20) = 0 - (-1) = 1$.
- קודקוד 25: הבן השמאלי 20 הוא כעת שורש של תת-עץ בגובה 1, והימני 30 הוא בגובה 0 $\implies \text{BF}(25) = 1 - 0 = 1$.
- קודקוד 15: תת-העץ השמאלי (שורשו 10) בגובה 1, ותת-העץ הימני (שורשו 25) הוא כעת בגובה 2 (בגלל תוספת 18) $\implies \text{BF}(15) = 1 - 2 = -1$.

העץ לאחר הכנסה ולפני תיקון:

```
      [15] (BF = -1)
     /    \
  [10]    [25] (BF = 1)
  /      /    \
[5]    [20]   [30] (BF = 1)
       /
     [18]
```

נבחן את גורמי האיזון:
כל גורמי האיזון בעץ הם בטווח התקין של עצי AVL $(|-1|, |1|, |0| \leq 1)$.
הדבר אומר ש**לא נוצרה שום הפרת איזון בעץ בעקבות ההכנסה של 18**, ולכן לא נדרשת אף רוטציה! העץ נשאר עץ AVL תקין כפי שהוא.

> ⚠ **מלכודת נפוצה! הפרות מדומות בעצי AVL**
>
> במבחנים רבים קיימות שאלות מכשיל הכוללות הכנסת איבר שאינה גוררת הפרת איזון כלל (כלומר גורמי האיזון של כל האבות במסלול נשארים בטווח של בין -1 ל-1). אין לבצע גלגולים באופן אוטומטי ללא חישוב מדויק של גורמי האיזון לאורך כל מסלול החיפוש מהעלה החדש ועד לשורש.

שאלה 7: גיבוב מושלם FKS

קבוצת מפתחות סטטית S בגודל $n = 4$ צריכה להיות מגובבת לטבלת גיבוב מושלמת דו-שלבית (אלגוריתם FKS).
נסמן ב-n_i את מספר המפתחות הממפים לתא i בטבלה הראשית (שגודלה $m = n = 4$).
נניח כי בהגרלת פונקציית הגיבוב הראשית התקבלו ערכי המיפוי הבאים עבור ארבעת המפתחות:

- מפתח k_1 גובב לתא 0.
- מפתחות k_2, k_3, k_4 גובבו כולם לתא 2.

1. מהו גודל כל אחת מהטבלאות המשניות S_i עבור התאים $i \in \{0, 1, 2, 3\}$?
2. מהו נפח הזיכרון הכולל שדורשות הטבלאות המשניות בשלב השני של האלגוריתם?

פתרון מפורט:

1. חישוב גודל הטבלאות המשניות:

על פי אלגוריתם FKS לגיבוב מושלם דו-שלבי:

- עבור תא i בטבלה הראשית שמכיל n_i מפתחות, אנו מקימים טבלת גיבוב משנית בגודל m_i השווה לריבוע מספר המפתחות שהגיעו אליו:

$$m_i = n_i^2$$

נחשב את ערכי n_i עבור כל תא לפי נתוני המיפוי:

- תא 0: מכיל את המפתח $k_1 \implies n_0 = 1$.
- תא 1: אינו מכיל מפתחות כלל $\implies n_1 = 0$.
- תא 2: מכיל את המפתחות $k_2, k_3, k_4 \implies n_2 = 3$.
- תא 3: אינו מכיל מפתחות כלל $\implies n_3 = 0$.

נחשב את גודל הטבלאות המשניות m_i:

- תא 0: $m_0 = n_0^2 = 1^2 = 1$.
- תא 1: $m_1 = n_1^2 = 0^2 = 0$.
- תא 2: $m_2 = n_2^2 = 3^2 = 9$.

- תא 3: $m_3 = n_3^2 = 0^2 = 0$.

2. חישוב נפח הזיכרון הכולל של הטבלאות המשניות:

נפח הזיכרון הכולל של השלב השני מוגדר כסכום הגדלים של כל הטבלאות המשניות שנוצרו:

$$\text{Total_Space} = \sum_{i=0}^{m-1} m_i = m_0 + m_1 + m_2 + m_3 = 1 + 0 + 9 + 0 = 10$$

נפח הזיכרון הכולל שנדרש עבור הטבלאות המשניות הוא 10 תאים.

שאלה 8: תכנון דינמי - תת-סדרה עולה ארוכה ביותר (LIS)

נתונה סדרת המספרים הבאה:

$$X = [10, 22, 9, 33, 21, 50, 41, 60]$$

1. בנה את מערך ה-DP התואם לחישוב אורך ה-LIS של X (על פי הגדרת תת-הבעיה המסתיימת באיבר x_i).
2. מהו אורך ה-LIS הכולל של X, ומהי תת-הסדרה עצמה?

פתרון מפורט:

1. בניית מערך ה-DP:

נגדיר את $F[i]$ להיות אורך תת-הסדרה העולה הארוכה ביותר המסתיימת בהכרח באיבר $X[i]$ (באינדוקס based-0 מ-0 עד 7).
נוסחת הנסיגה:

$$F[i] = 1 + \max(\{F[j] \mid 0 \le j < i \text{ and } X[j] < X[i]\} \cup \{0\})$$

נחשב את ערכי המערך צעד אחר צעד:

- $i = 0$ **($X[0] = 10$):** אין איברים קודמים $\implies F[0] = 1$.
- $i = 1$ **($X[1] = 22$):**
  - איברים קודמים קטנים מ-22: $\{X[0] = 10\}$ (ערך ה-DP שלו הוא $F[0] = 1$).
  - $F[1] = 1 + F[0] = 1 + 1 = 2$.
- $i = 2$ **($X[2] = 9$):**
  - אין איברים קודמים קטנים מ-9 $\implies F[2] = 1$.
- $i = 3$ **($X[3] = 33$):**
  - איברים קודמים קטנים מ-33: $\{X[0] = 10, X[1] = 22, X[2] = 9\}$ (ערכי ה-DP שלהם: $\{1, 2, 1\}$).
  - המקסימום מביניהם הוא 2 (שהושג ב-$X[1]$).
  - $F[3] = 1 + 2 = 3$.
- $i = 4$ **($X[4] = 21$):**
  - איברים קודמים קטנים מ-21: $\{X[0] = 10, X[2] = 9\}$ (ערכי ה-DP שלהם: $\{1, 1\}$).
  - המקסימום הוא 1.
  - $F[4] = 1 + 1 = 2$.
- $i = 5$ **($X[5] = 50$):**
  - איברים קודמים קטנים מ-50: כל האיברים הקודמים $\{10, 22, 9, 33, 21\}$ (ערכי ה-DP שלהם: $\{1, 2, 1, 3, 2\}$).
  - המקסימום הוא 3 (שהושג ב-$X[3]$).
  - $F[5] = 1 + 3 = 4$.
- $i = 6$ **($X[6] = 41$):**
  - איברים קודמים קטנים מ-41: $\{10, 22, 9, 33, 21\}$ (ערכי ה-DP שלהם: $\{1, 2, 1, 3, 2\}$).
  - המקסימום הוא 3 (שהושג ב-$X[3]$).
  - $F[6] = 1 + 3 = 4$.
- $i = 7$ **($X[7] = 60$):**
  - איברים קודמים קטנים מ-60: כל האיברים הקודמים (ערכי ה-DP: $\{1, 2, 1, 3, 2, 4, 4\}$).
  - המקסימום הוא 4 (שהושג ב-$X[5]$ או $X[6]$).
  - $F[7] = 1 + 4 = 5$.

מערך ה-DP שהתקבל:

$$F = [1, 2, 1, 3, 2, 4, 4, 5]$$

2. אורך ה-LIS ושחזור הסדרה:

- **אורך ה-LIS הכללי:** הערך המקסימלי במערך F, שהוא:

$$\max(F) = 5$$

- **שחזור הסדרה (מלמטה למעלה באמצעות מעקב לאחור):**
  - נתחיל מאינדקס 7 ($X[7] = 60$), שערכו $F[7] = 5$.
  - נחפש איבר קודם $X[j]$ המקיים $X[j] < 60$ ועבורו $F[j] = 5 - 1 = 4$. האינדקסים המתאימים הם $j = 5$ ($X[5] = 50$) או $j = 6$ ($X[6] = 41$). נבחר ב-$j = 6$ (ערכו 41).
  - נחפש איבר קודם $X[j] < 41$ שעבורו $F[j] = 4 - 1 = 3$. האינדקס המתאים הוא $j = 3$ ($X[3] = 33$).
  - נחפש איבר קודם $X[j] < 33$ שעבורו $F[j] = 3 - 1 = 2$. האינדקסים המתאימים הם $j = 1$ ($X[1] = 22$) או $j = 4$ ($X[4] = 21$). נבחר ב-$j = 1$ (ערכו 22).
  - נחפש איבר קודם $X[j] < 22$ שעבורו $F[j] = 2 - 1 = 1$. האינדקס המתאים הוא $j = 0$ ($X[0] = 10$).

  שחזור הסדרה נותן את ה-LIS הבא:

$$[10, 22, 33, 41, 60]$$

(הערה: קיימת סדרה אופטימלית נוספת באותו אורך שהיא $[10, 22, 33, 50, 60]$).

שאלות ברמה קשה (Advanced/Hard)

שאלה 9: תכנון דינמי - וריאציית משחק השטרות

שני שחקנים משחקים במשחק השטרות המוכר על גבי שורת שטרות $v_1, \ldots, v_n \geq 0$.
הפעם חוקי המשחק משתנים: בכל תור, השחקן שזהו תורו רשאי לקחת שטר בודד מקצוות הלוח (השמאלי או הימני), **או לחילופין לוותר לחלוטין על לקיחת שטר בתורו** (במקרה כזה, התור עובר ישירות לשחקן השני מבלי ששולבו שטרות לקופת השחקן הנוכחי).
לא ניתן לבצע שני ויתורים רצופים במשחק (כלומר, אם שחקן א' בחר לוותר בתורו, שחקן ב' מחויב לבצע לקיחה פיזית של שטר ואינו רשאי לוותר).
פתח נוסחת נסיגה בתכנון דינמי המחשבת את סכום הכסף המקסימלי שהשחקן הראשון יכול להבטיח לעצמו תחת חוקים אלו.

פתרון מפורט:

בשל חוק מניעת שני ויתורים רצופים, מצב המשחק אינו נקבע רק על ידי קטע השטרות שנותר על הלוח $[i, j]$, אלא עלינו לשמור שדה מצב בוליאני המציין האם השחקן הקודם ביצע ויתור.
נגדיר שתי פונקציות DP על קטע הלוח $v_i, \ldots, v_j$ (עבור $1 \leq i \leq j \leq n$):

1. $p_1(i, j)$**:** הסכום המקסימלי שהשחקן שזהו תורו כעת יכול להבטיח לעצמו, בהינתן ש**מותר לו לוותר** בתור הנוכחי (כלומר, השחקן הקודם ביצע לקיחה פיזית של שטר).
2. $p_0(i, j)$**:** הסכום המקסימלי שהשחקן שזהו תורו כעת יכול להבטיח לעצמו, בהינתן ש**אסור לו לוותר** בתור הנוכחי (כלומר, השחקן הקודם ביצע ויתור).

נפתח את נוסחאות הנסיגה עבור שתי הפונקציות באמצעות עקרון המינימקס ומשחקי סכום אפס:
סכום השטרות בקטע הוא $S(i, j) = \sum_{k=i}^{j} v_k$.

א. פיתוח נוסחת הנסיגה עבור $p_0(i, j)$ (אסור לוותר):

מאחר שאסור לוותר, לשחקן יש בדיוק שתי בחירות פיזיות (שמאל או ימין):

- **אם ייקח את השמאלי (v_i):** היריב מקבל את הקטע שנותר $i + 1, j$, ומאחר שביצענו כעת לקיחה פיזית, ליריב מותר לוותר בתורו. לכן היריב יפיק מקטע זה שווי של $p_1(i + 1, j)$. השווי שנותר לנו לקבל מתוך הקטע הוא: $S(i + 1, j) - p_1(i + 1, j)$.
  הרווח הכולל במקרה זה הוא: $v_i + S(i + 1, j) - p_1(i + 1, j) = S(i, j) - p_1(i + 1, j)$.
- **אם ייקח את הימני (v_j):** בניתוח סימטרי, הרווח הכולל שנקבל הוא: $S(i, j) - p_1(i, j - 1)$.

נבחר באפשרות המקסימלית:

$$p_0(i, j) = S(i, j) - \min\left\{p_1(i + 1, j),\ p_1(i, j - 1)\right\}$$

ב. פיתוח נוסחת הנסיגה עבור $p_1(i, j)$ (מותר לוותר):

כאן לשחקן יש שלוש בחירות אפשריות: לקיחה משמאל, לקיחה מימין, או ביצוע ויתור.

- אם השחקן בוחר לבצע לקיחה פיזית (שמאל או ימין), אנו חוזרים בדיוק לאפשרויות של $p_0(i, j)$, שהרווח המקסימלי מביניהן הוא $p_0(i, j)$.

- אם השחקן בוחר לבצע **ויתור**: השחקן אינו מקבל שום שטר באופן מיידי. הלוח נשאר ללא שינוי $[i,j]$. התור עובר ליריב, ומאחר שביצענו כעת ויתור, ליריב אסור לוותר בתורו, ולכן היריב יפיק מהלוח שווי של $p_0(i,j)$. הרווח שנותר לנו לקבל מהלוח תחת בחירה זו הוא:

$$S(i,j) - p_0(i,j)$$

נבחר באפשרות המקסימלית מבין ביצוע הלקיחה הפיזית לבין ביצוע הויתור:

$$p_1(i,j) = \max\left\{p_0(i,j),\ S(i,j) - p_0(i,j)\right\}$$

ג. מקרי בסיס (עבור שטר בודד $i = j$):

- **עבור $p_0(i,i)$ (אסור לוותר):** השחקן חייב לקחת את השטר היחיד:

$$p_0(i,i) = v_i$$

- **עבור $p_1(i,i)$ (מותר לוותר):**
  - אם השחקן לוקח, הוא מקבל v_i.
  - אם השחקן מוותר, היריב מקבל את השטר v_i (כי אסור ליריב לוותר כעת), ולכן אנו מקבלים 0.
  מאחר ש-$v_i \geq 0$, השחקן תמיד יעדיף לקחת ולא לוותר:

$$p_1(i,i) = \max\{v_i,\ 0\} = v_i$$

- **פתרון הבעיה המקורית עבור לוח שלם $v_1 \dots v_n$:**
בתחילת המשחק מותר לשחקן הראשון לבצע ויתור (מכיוון שטרם בוצעו צעדים), ולכן התשובה תהיה:

$$p_1(1,n)$$

אסטרטגיית פתרון: סדר החישוב

מילוי שתי הטבלאות p_0 ו-p_1 בגודל $n \times n$ יתבצע במקביל, לפי סדר גודל קטע עולה $l = j - i + 1$ מ-2 ועד n. לצורך חישוב הטווחים $S(i,j)$ ב-$O(1)$, נבצע עיבוד מקדים של מערך סכומי רישות $S[t]$. זמן הריצה הכללי יהיה $O(n^2)$ וסיבוכיות המקום $O(n^2)$.

שאלה 10: מבנה קבוצות זרות (Union-Find) עם משקלים

עצב גרסה משופרת למבנה הנתונים קבוצות זרות (Union-Find) התומכת בפעולות הבאות:

- `MakeSet(x)` : יצירת קבוצה המכילה את x.
- `Union(x, y)` : איחוד הקבוצות של x ו-y.
- `Find(x)` : מציאת נציג הקבוצה של x.
- `FindMax(x)` : החזרת הערך של **האיבר המקסימלי** הקיים כיום בקבוצה שאליה שייך האיבר x.
ירוצו בזמן של לכל היותר $O(\alpha(n))$ לשיעורין (כאשר $\alpha(n)$ היא פונקציית אקרמן ההפוכה) (`FindMax` כולל) דרוש כי כל פעולות המבנה.

פתרון מפורט:

נממש את המבנה באמצעות יער של עצים הפוכים, תוך שימוש בשני השיפורים המקבילים: **איחוד לפי דרגה (Union by Rank)** ו**כיווץ מסלולים (Path Compression)**.

ארכיטקטורת שדות הנתונים בכל קודקוד v:

לכל קודקוד v ננהל את השדות הבאים:

- `v.parent` : מצביע לאב של v בעץ.
- `v.rank` : חסם עליון לגובה העץ (עבור קודקוד שהוא שורש).
- `v.max_val` : הערך המקסימלי הקיים בקבוצה (עבור קודקוד שהוא שורש/נציג הקבוצה).

מימוש הפעולות:

1. פעולת `MakeSet(x)` :

ניצור קודקוד חדש עבור x:

```
def make_set(v):
    v.parent = v
    v.rank = 0
    v.max_val = v.val  # בהתחלה האיבר היחיד הוא המקסימום
```

סיבוכיות זמן: $O(1)$.

2. פעולת `Find(x)` ופעולת `FindMax(x)`:

פעולת ה- `Find(x)` הרגילה מבוצעת עם כיווץ מסלולים:

```
def find(v):
    if v.parent != v:
        v.parent = find(v.parent)  # כיווץ מסלולים
    return v.parent
```

כדי לממש את `FindMax(x)` ביעילות, נשים לב שהערך המקסימלי המעודכן של כל קבוצה שמור תמיד ב**שורש העץ** של הקבוצה. לכן, כדי למצוא אותו, נמצא תחילה את השורש של x באמצעות `Find(x)`, ונשלוף את שדה ה- `max_val` שלו:

```
def find_max(v):
    root = find(v)  # מציאת נציג הקבוצה
    return root.max_val
```

סיבוכיות זמן של `Find` ושל `FindMax` היא $O(\alpha(n))$ לשיעורין, עקב השימוש בכיווץ מסלולים.

3. פעולת `Union(x, y)`:

נמצא את נציגי הקבוצות של x ו-y:

```
def union(x, y):
    root_x = find(x)
    root_y = find(y)

    if root_x == root_y:
        return  # כבר באותה קבוצה

    # איחוד לפי דרגה
    if root_x.rank < root_y.rank:
        root_x.parent = root_y
        # עדכון הערך המקסימלי בשורש החדש
        root_y.max_val = max(root_y.max_val, root_x.max_val)
    else:
        root_y.parent = root_x
        if root_x.rank == root_y.rank:
            root_x.rank += 1
        # עדכון הערך המקסימלי בשורש החדש
        root_x.max_val = max(root_x.max_val, root_y.max_val)
```

הוכחת יעילות וסיבוכיות:

- **MakeSet**: דורשת הקצאה ועדכון שדות ב-$O(1)$.
- **Union**: בשורש הנבחר על ידי חישוב מקסימום בין שני ערכים קבועים - `max_val` מבצעת שתי קריאות ל- `Find`, ועדכון שדה ה- `max_val` השוואת דרגות, ועדכון שדה ה- `max_val` (פעולה ב-$O(1)$).
- **Find ו-FindMax**: משתמשת באיחוד לפי דרגה. שילוב שני שיפורים אלו מבטיח על פי `Union` משתמשת בכיווץ מסלולים, ופעולת `Find` פעולת $O(m \cdot \alpha(n))$. כי סך זמן הריצה של סדרת m פעולות במבנה נתונים בעל n איברים חסום על ידי Tarjan המשפט הידוע של
מכאן שסיבוכיות הפעולות לשיעורין היא $O(\alpha(n))$ כמבוקש.

שאלה 11: אלגוריתם אימות עץ AVL בשידור ליניארי

כתוב אלגוריתם המקבל עץ בינארי כללי T (לא בהכרח BST) וקובע האם T הוא עץ AVL תקין (כלומר, מקיים את תכונת עץ חיפוש בינארי BST, וכן את תנאי האיזון של AVL לכל קודקוד).
דרוש כי האלגוריתם ירוץ בזמן ליניארי של $O(n)$ במקרה הגרוע ביותר, וישתמש בתוספת מקום של לכל היותר $O(h)$ (עבור מחסנית הרקורסיה בלבד, ללא הקצאת זיכרון דינמי נוסף).

פתרון מפורט:

כדי לאמת את העץ בזמן ליניארי $O(n)$, עלינו להימנע מחישוב גבהים נפרד לכל קודקוד (שיעלה את הסיבוכיות ל-$O(n \log n)$ או $O(n^2)$).
במקום זאת, נתכנן פונקציה רקורסיבית אחת המחשבת עבור כל קודקוד v שלושה פרמטרים במקביל במהלך סריקה מאוחרת (Post-Order):

1. האם תת-העץ של v הוא AVL תקין (ערך בוליאני).
2. גובה תת-העץ של v.
3. הערך המינימלי והערך המקסימלי הקיים בתת-העץ של v (לצורך אימות תכונת ה-BST).

אלגוריתם ופסאודו-קוד:

```
def validate_avl(root):
    # הפונקציה מחזירה רביעייה: (is_avl, height, min_val, max_val)
    def check_node(v):
        if v is None:
            # מקרה בסיס עבור עץ ריק
            # הגדרת אינסוף עבור ערכים מאפשרת לכל ערך לעבור את תנאי ה-BST
            return (True, -1, float('inf'), float('-inf'))

        # סריקה רקורסיבית של שני הבנים (מלמטה למעלה)
        left_valid, left_h, left_min, left_max = check_node(v.left)
        right_valid, right_h, right_min, right_max = check_node(v.right)

        # 1. בדיקה ששני תתי-העצים תקינים בעצמם
        if not left_valid or not right_valid:
            return (False, 0, 0, 0)

        # 2. אימות תכונת BST עבור הקודקוד הנוכחי v:
        # חייב להיות גדול ממש מהערך המקסימלי בתת-העץ השמאלי של v המפתח
        # וקטן ממש מהערך המינימלי בתת-העץ הימני.
        if v.val <= left_max or v.val >= right_min:
            return (False, 0, 0, 0)

        # 3. אימות תנאי האיזון AVL עבור הקודקוד הנוכחי v:
        # הפרש הגבהים בין הבנים חייב להיות לכל היותר 1
        if abs(left_h - right_h) > 1:
            return (False, 0, 0, 0)

        # 4. חישוב הפרמטרים עבור הקודקוד הנוכחי:
        curr_height = 1 + max(left_h, right_h)
        # המינימום בתת-העץ הוא המינימום של הבן השמאלי (או v עצמו, אם אין בן שמאלי)
        curr_min = min(v.val, left_min)
        # המקסימום בתת-העץ הוא המינימום של הבן הימני (או v עצמו, אם אין בן ימני)
        curr_max = max(v.val, right_max)

        return (True, curr_height, curr_min, curr_max)

    # הרצה מהשורש
    is_avl, _, _, _ = check_node(root)
    return is_avl
```

ניתוח סיבוכיות:

- **סיבוכיות זמן:** $O(n)$ במקרה הגרוע ביותר. האלגוריתם מבצע סריקת Post-Order רקורסיבית יחידה על פני כל n הקודקודים בעץ. העבודה המבוצעת בכל קודקוד מורכבת רק מהשוואות אריתמטיות פשוטות וחישובי מינימום/מקסימום בין ערכים קבועים, פעולה הלוקחת זמן קבוע של $O(1)$.
- **סיבוכיות מקום:** $O(h)$ במקרה הגרוע ביותר (כאשר h הוא גובה העץ). הזיכרון הנוסף היחיד שנעשה בו שימוש הוא מחסנית הקריאות של הרקורסיה. לא הוקצו מבני נתונים דינמיים נוספים.

שאלה 12: ניתוח פיצולים במקרה הגרוע בעץ B-Tree

נתון עץ B-Tree ריק מסדר m (כאשר m הוא זוגי). אנו מכניסים לעץ סדרה של n מפתחות בזה אחר זה.
הוכח כי מספר פעולות הפיצול (Split) הכולל המבוצע במהלך כל n ההכנסות חסום מלמעלה על ידי:

$$\text{Total_Splits} \leq \frac{n-1}{\lceil m/2 \rceil - 1}$$

פתרון מפורט:

נוכיח את החסם באמצעות **שיטת הצבירה (Aggregate Method)** או על ידי מעקב אחר מספר המפתחות המאוחסנים בעץ.

ניתוח תוספת המפתחות בכל פעולת פיצול:

נבחן כמה מפתחות קיימים בעץ כפונקציה של מספר הקודקודים:

1. בעץ B-Tree מסדר m, כל קודקוד (פרט לשורש) מחזיק לפחות $t-1$ מפתחות, כאשר דרגת המינימום היא $t = \lceil m/2 \rceil$.
2. השורש של העץ רשאי להחזיק לפחות מפתח בודד (כאשר העץ אינו ריק).
3. כאשר קודקוד כלשהו בעץ עובר פיצול (Split):
   - הקודקוד הכיל לפני הפיצול בדיוק $m-1$ מפתחות (קודקוד מלא גלישה).
   - במהלך הפיצול, מפתח החציון עולה למעלה לאב הקודקוד.
   - שאר $m-2$ המפתחות מתחלקים שווה בשווה בין שני קודקודים חדשים שנוצרים: כל אחד מהם מקבל בדיוק $\frac{m-2}{2}$ או $\frac{m-1}{2}$ מפתחות (אם m אי-זוגי). מאחר ש-m זוגי, מתקיים $\lceil m/2 \rceil = m/2$. לכן, כל אחד משני הקודקודים החדשים מקבל בדיוק:

$$\frac{m-2}{2} = \frac{m}{2} - 1 = t - 1$$

מפתחות.

נבחין כי כל פעולת פיצול (למעט פיצול של השורש המגדיל את גובה העץ) מייצרת **קודקוד חדש אחד בעץ**.

- קודקוד חדש זה חייב להכיל לפחות $t-1$ מפתחות.
- מפתחות אלו הועברו אליו מתוך הקודקוד המלא שהתפצל, ולכן הם מפתחות שכבר היו קיימים בעץ.
עם זאת, כדי שקודקוד כלשהו בעץ יתמלא מחדש ויעבור פיצול נוסף בעתיד, יש להכניס מפתחות חדשים לעץ שיזרמו אליו במהלך פעולות ה-`Insert`.

נסמן ב-N_{nodes} את מספר הקודקודים בעץ לאחר הכנסת n מפתחות.

- השורש מכיל לפחות מפתח 1.
- שאר $N_{nodes} - 1$ הקודקודים מכילים לפחות $t-1$ מפתחות כל אחד.
סך כל המפתחות בעץ n חייב לקיים את האי-שוויון הבא:

$$n \geq 1 + (N_{nodes} - 1) \cdot (t-1)$$

נחלץ מתוך האי-שוויון את מספר הקודקודים N_{nodes}:

$$n - 1 \geq (N_{nodes} - 1) \cdot (t-1) \implies N_{nodes} - 1 \leq \frac{n-1}{t-1}$$

$$N_{nodes} \leq 1 + \frac{n-1}{t-1}$$

קשר למספר הפיצולים:

- אנו מתחילים מעץ ריק המכיל קודקוד בודד (השורש, $N_{nodes} = 1$).
- כל פעולת פיצול (Split) מייצרת בדיוק קודקוד חדש אחד בעץ (הפיצול מפצל קודקוד לשניים, כלומר מוסיף קודקוד אחד).
- מכאן שמספר הפיצולים הכולל שבוצע, המסומן ב-S_{total}, שווה בדיוק למספר הקודקודים שהתווספו לעץ, כלומר:

$$S_{total} = N_{nodes} - 1$$

נציב את החסם שקיבלנו עבור N_{nodes}:

$$S_{total} \leq \left(1 + \frac{n-1}{t-1}\right) - 1 = \frac{n-1}{t-1}$$

נציב בחזרה את $t = \lceil m/2 \rceil$:

$$S_{total} \leq \frac{n-1}{\lceil m/2 \rceil - 1}$$

הוכחנו את החסם.

משפט: מספר פיצולים ממוצע קבוע

על פי החסם שהוכחנו, מספר הפיצולים הממוצע לפעולת הכנסה יחידה (לשיעורין) חסום על ידי:

$$\frac{S_{total}}{n} \leq \frac{n-1}{n(\lceil m/2 \rceil - 1)} < \frac{1}{\lceil m/2 \rceil - 1}$$

עבור עץ B-Tree מדרגה קבועה (למשל $m = 100$), ערך זה קטן מאוד (פחות מ-$1/49$), מה שמבטיח שפעולות פיצול מתרחשות לעיתים נדירות מאוד, והעלות לשיעורין של פיצולים בהכנסה היא $O(1)$ קבועה.